CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY (CHARUSAT)

Chandubhai S. Patel Institute of Technology (CSPIT)

Department of Computer Science & Engineering

ITUE203 – Web Development Frameworks

PRACTICAL 13

Event Finder – AJAX Event CRUD with the Fetch API and a PHP JSON API

PHP 8 · MySQLi · MariaDB (studenthub_db) · JavaScript Fetch API · JSON

Corrected submission – implemented in the integrated StudentHub portal (studenthub/)

Student Name	Shreyas
Roll Number	25CS023
Student ID	25CS023
Email	25cs023@charusat.edu.in
Course	ITUE203 – Web Development Frameworks
Programme	B.Tech Computer Science & Engineering
Semester / Academic Year	3 / 2026-27
Date of Submission	
Faculty Signature	

Corrected report. This report replaces the original Practical 13 report (`Practical-13/PRAC_13_ITUE203_25CS023.docx`, kept unchanged in the repository). The original report describes a standalone application in `Practical-13/` (`index.html`, `script.js`, `api.php`) with its own database `studenthub_prac13`. This corrected version describes the Event Finder of the integrated portal: `api/events.php` uses the portal's database connection, login session, roles and CSRF token and the same event and registration rules as the Admin Console (Practical 12); no schema change was needed. All screenshots in this report were taken from the canonical integrated application `studenthub/` with Microsoft Edge on disposable test databases with fictional records; all test results are copied from the saved test outputs of the repository.

1. Aim and Objectives

To manage campus events asynchronously inside the integrated portal: the browser sends small requests with the JavaScript Fetch API to a PHP endpoint that answers in JSON, and the page updates only the affected part – it is never reloaded.

- List events from the portal database and – for administrators – add, edit/update and delete events with Fetch requests to `api/events.php`, without reloading the page.
- Return valid JSON for every request with a fitting status code, handle GET (read) and POST (change) correctly and answer other methods with 405.
- Validate all input on the server before any database operation (the same `validate_event()` as the Admin Console), use MySQLi prepared statements and output data safely.
- Handle errors centrally in JSON and show loading, success, empty and error states in the page.
- Intermediate extension: an AJAX search with debounce (request sent 300 ms after the last key press) combined with category, registration-status and date filters.
- Integration: respect the portal login and roles – students with a linked student record register for events and cancel through the same API; POST requests carry the session's CSRF token.

2. Problem Definition

In a classic PHP application every action submits a form and reloads the whole page. An event list that is searched and edited often feels slow that way and loses the user's position and search. A Fetch-based page avoids this, but moves the attack surface to a JSON endpoint that any script can call directly. The endpoint therefore has to make every decision itself: who is signed in, which role may do what, whether the request really comes from the portal page (CSRF), whether the body is valid JSON and whether the event data is valid.

In the standalone version the API had its own database `studenthub_prac13` and its own event table and protected POST requests with the header `X-Requested-With: fetch`. In the portal the same events are also managed in the Admin Console (Practical 12) and registered for on the Campus Events page, so the API must use the same tables, the same validation and registration rules, the portal session and roles – and it must answer a missing session with JSON HTTP 401 instead of the HTML redirect to the login page that normal portal pages use.

3. Tools and Technologies

Tool	Use in this practical
PHP 8.2.12 (XAMPP), MySQLi	JSON API `api/events.php`, prepared statements, central JSON error handling
MariaDB 10.4.32 (XAMPP)	Shared database `studenthub_db` (tables events, event_categories, registrations, students)

JavaScript (ES2020) – Fetch API	`fetch()`, async/await, `AbortController`, `URLSearchParams`, `FormData`, JSON
HTML5 `<dialog>`, CSS3	`events_live.php` (filters, result list, modal create/edit dialog), styles in `assets/css/global.css`
Microsoft Edge (headless) + Playwright	`tests/browser_api.py` – browser checks, request and navigation counting, screenshots
Python 3	`tests/test_api.py` – HTTP tests of every API action on disposable databases
Git / GitHub	Commit `a9d4e22`, pushed to the project repository

4. Place in the Integrated Portal and Source Files

The Event Finder is a normal portal page (sidebar group *Campus → Event Finder*) that every signed-in user can open. The five required files of the original practical map to these portal files:

Original file	Portal file	Purpose
db.php	`studenthub/db.php`	The portal's MySQLi connection; with `STUDENTHUB_API` defined it answers its own failures (HTTP 503) as JSON
api.php	`studenthub/api/events.php`	JSON API: list, get (GET); create, update, delete, register, cancel (POST)
index.html	`studenthub/events_live.php`	Event Finder page: CSRF meta tag, filters, result list, loading indicator, messages, create/edit dialog for administrators
script.js	`studenthub/assets/js/events-live.js`	Fetch front end: `api()` wrapper, rendering with `textContent`, debounced search, dialog, register / cancel
schema.sql	`database/studenthub_demo.sql` + migrations	No schema change for Practical 13; the tables of Practicals 8, 11 and 12 are used

The API reuses the portal's own code instead of duplicating it: `includes/auth.php` (session, `role_can()`, `csrf_is_valid()`), `includes/event_management.php` (filters, `validate_event()`, `save_event()`, `delete_event()`, `delete_poster_file()`), `includes/event_registration.php` (`register_student_for_event()`, `cancel_event_registration()`) and `audit_logger.php`. What each role can do:

Role	Event Finder page	API actions allowed
admin	New event button, Edit and Delete on every card	list, get, create, update, delete (`events.manage`); register / cancel → 403
student with a linked record	Register / Cancel registration	list, get, register, cancel; create / update / delete → 403 (ACCESS_DENIED logged)
editor	View and search only	list, get
not signed in	Redirect to the login page	every action → 401 JSON, never a redirect

5. Database Schema

Practical 13 needed no schema change (TEST_REPORT §10). The API reads and writes the tables described in the Practical 12 report: `events` (with `status`, `capacity` and the unique `poster` from migrations 001 and 003), `event_categories` (allowed categories for the filter and the validation), `registrations` (each student's own registration state; UNIQUE student + event) and `students` (the record linked to the signed-in account through `students.user_id`). The seats taken and the number of registrations are counted by sub-

queries in the same SELECT. `api_event()` turns a row into JSON with exactly these fields – internal columns and file names are never sent:

JSON field	Meaning
id, title, category, location, description	Event data; <, >, & and quotes are sent as \u escapes, and the page inserts the values with `textContent`
event_date, event_time, when	Date `YYYY-MM-DD`, time `HH:MM` or null, and a display label such as "18 Nov 2026, 10:00"
capacity, seats_taken, seats_left	Capacity null = no limit; seats_left null when there is no limit
status, is_past	`open` / `closed`; whether the date has passed
poster_url	`poster.php?event=N` when a stored poster exists, otherwise null
registrations_total, my_registration	All registrations of the event; the signed-in student's own state (registered / attended / cancelled / null)

6. AJAX CRUD Implementation

6.1 PHP JSON API (api/events.php)

`api/events.php` reads `action` from the query string. `list` and `get` require GET; `create`, `update`, `delete`, `register` and `cancel` require POST with a JSON body. Every answer leaves through one function, `api_respond()`, which sets the status code, `Content-Type: application/json`, `Cache-Control: no-store` and `nosniff`, and encodes the body with `JSON_HEX_TAG | JSON_HEX_AMP | JSON_HEX_APOS | JSON_HEX_QUOT`, so <, >, & and quotes never appear literally in the JSON text. Expected problems are thrown as `ApiError` with a status code; anything else is caught by the central exception handler, written to the PHP error log and answered with a general 500 message – no SQL or connection details reach the browser. PHP warnings are turned into exceptions, so they take the same path.

studenthub/api/events.php (lines 31–48) – single JSON exit point and central error handling

```
31  // The single exit point: status code, JSON headers and the JSON body
32  function api_respond(int $status, array $body): void
33  {
34      http_response_code($status);
35      header('Content-Type: application/json; charset=utf-8');
36      header('Cache-Control: no-store');
37      header('X-Content-Type-Options: nosniff');
38      echo json_encode($body, JSON_UNESCAPED_UNICODE | JSON_UNESCAPED_SLASHES | JSON_HEX_TAG |
    JSON_HEX_AMP | JSON_HEX_APOS | JSON_HEX_QUOT);
39      exit;
40  }
41
42  // Central error handling: expected problems are ApiError; anything else is logged and answered with
    a general 500
43  set_exception_handler(function (Throwable $e): void {
44      if ($e instanceof ApiError) {
45          api_respond($e->getCode(), ['ok' => false, 'error' => $e->getMessage()] + ($e->fields ?
    ['errors' => $e->fields] : []));
46      }
47      error_log('StudentHub API error: ' . get_class($e) . ' [' . $e->getCode() . '] ' . $e-
    >getMessage() . ' in ' . $e->getFile() . ':' . $e->getLine());
48      api_respond(500, ['ok' => false, 'error' => 'The server could not complete the request. Please
    try again later.']);
```

For every POST, `json_body()` checks – in this order – the content type (415 unless `application/json`), the CSRF token in the `X-CSRF-Token` header (403, logged as CSRF_REJECTED), the size of the body (400 above 20,000 bytes) and that the body is a JSON object (400 for malformed JSON or a JSON array). A cross-site HTML form can send neither `application/json` nor a custom header, and it cannot know the session's token.

studenthub/api/events.php (lines 62–82) – request checks for POST

```
// POST: JSON body, CSRF header; returns the decoded object
function json_body(array $user): array
{
    global $conn;
    $type = strtolower(trim(explode(';', (string) ($_SERVER['CONTENT_TYPE'] ?? ''))[0]));
    if ($type !== 'application/json') {
        throw new ApiError(415, 'Send the request body as JSON (Content-Type: application/json).');
    }
    if (!csrf_is_valid($_SERVER['HTTP_X_CSRF_TOKEN'] ?? null)) {
        log_audit($conn, $user['id'], $user['username'], 'CSRF_REJECTED', 'API POST ' .
($_GET['action'] ?? '') . ' without a valid token', 'SECURITY_BLOCK');
        throw new ApiError(403, 'Security check failed. Reload the page and try again.');
    }
    $raw = file_get_contents('php://input');
    if (strlen((string) $raw) > 20000) {
        throw new ApiError(400, 'The request body is too large.');
    }
    $data = json_decode((string) $raw, true);
    if (!is_array($data) || array_is_list($data) && $data !== []) {
        throw new ApiError(400, 'The request body must be a JSON object.');
    }
    return $data;
```

6.2 Session, roles and JSON 401

Normal portal pages call `require_login()`, which redirects to the login page. A redirect would be useless for `fetch()`, so the API starts the same secure session and calls `validate_session()` directly: without a valid session (never signed in, timed out, or a deactivated student) it answers HTTP 401 in JSON. Then `portal.view` is required, and each action checks its own permission (`events.manage` for create / update / delete; a linked student record for register / cancel).

studenthub/api/events.php (lines 133–147) – authentication without redirects

```
// ------------------------------------------------------------- authentication (no redirects:
JSON 401)
start_secure_session();
send_security_headers(true);
[$user, $reason] = validate_session();
if ($user === null) {
    throw new ApiError(401, match ($reason) {
        'timeout'         => 'Your session ended because you were inactive. Please sign in again.',
        'access_disabled' => 'Your access to StudentHub is currently disabled. Please contact the
administrator.',
        default           => 'Please sign in to use StudentHub.',
    });
}
require_api_permission($user, 'portal.view');
$categories = event_categories($conn);
$student = $user['role'] === 'student' ? linked_student($conn, $user['id']) : null;
$action = is_string($_GET['action'] ?? null) ? $_GET['action'] : '';
```

`list` passes the query parameters to the same `event_filters()` as the Admin Console (unknown values are ignored), returns at most 100 events and adds the role flags `can_manage` and `can_register` that the page uses to decide which buttons to show – the server still checks every action. `create` and `update` validate the JSON body with `validate_event()` and save it with `save_event()`, so every rule of Practical 12 applies here too, including the capacity that cannot drop below the seats taken. `delete` uses `delete_event()` and answers 409 for an event with registrations. `register` / `cancel` call the registration transaction and map its reasons to status codes (unknown event 404, inactive student 403, duplicate / full / closed / past / not registered 409).

studenthub/api/events.php (lines 168–185) – create and update with the shared validation

```
    case 'create':
    case 'update':
        require_method('POST');
        require_api_permission($user, 'events.manage');
        $body = json_body($user);
        $existing = null;
```

```
174         if ($action === 'update' && ($existing = ($id = body_id($body, 'id')) > 0 ? find_event($conn, $id) : null) === null) {
175             throw new ApiError(404, 'That event was not found.');
176         }
177         [$clean, $errors] = validate_event(event_input($body), $categories, $existing);
178         $saved = $errors ? ['errors' => $errors] : save_event($conn, $clean, $existing ? (int) $existing['id'] : null);
179         if (isset($saved['errors'])) {
180             throw new ApiError(422, 'Please correct the highlighted fields.', $saved['errors']);
181         }
182         log_audit($conn, $user['id'], $user['username'], $existing ? 'UPDATE_EVENT' : 'CREATE_EVENT',
183             ($existing ? 'Updated' : 'Created') . " event ID {$saved['id']} ({$clean['title']}) via the events API", 'SUCCESS');
184         api_respond($existing ? 200 : 201, ['ok' => true, 'event' => api_event(find_event($conn, $saved['id']), null),
185             'message' => $existing ? "Event “{$clean['title']}” was updated." : "Event “{$clean['title']}” was published."]);
```

6.3 Fetch API front end (events-live.js)

All requests go through `api()`. It builds the URL with `URLSearchParams`, sends JSON with the CSRF token (read from the page's `<meta name="csrf-token">`) for POST requests, parses the JSON answer and turns a network failure or a non-JSON answer into an error object, so the caller always receives `{status, data}`.

studenthub/assets/js/events-live.js (lines 45–69) – the Fetch wrapper

```
45      // One API call. Returns {status, data}; network failures and non-JSON answers become an error object.
46      async function api(action, { method = 'GET', params = {}, body = null, signal } = {}) {
47          const url = API + '?' + new URLSearchParams({ action, ...params });
48          const headers = { Accept: 'application/json' };
49          if (body !== null) {
50              headers['Content-Type'] = 'application/json';
51              headers['X-CSRF-Token'] = token;
52          }
53          let response;
54          try {
55              response = await fetch(url, { method, headers, body: body === null ? undefined : JSON.stringify(body), credentials: 'same-origin', signal });
56          } catch (error) {
57              if (error.name === 'AbortError') throw error;
58              return { status: 0, data: { ok: false, error: 'The server could not be reached. Check your connection and try again.' } };
59          }
60          let data;
61          try {
62              data = await response.json();
63          } catch {
64              data = { ok: false, error: 'The server sent an unexpected answer (HTTP ' + response.status + ').' };
65          }
66          if (response.status === 401) {
67              data.error = (data.error || 'Please sign in again.') + ' Reload the page to sign in.';
68          }
69          return { status: response.status, data };
```

`load()` reads the filter form, shows the loading indicator, marks the list `aria-busy` and cancels a still running older request with an `AbortController`, so results never arrive in the wrong order. The cards are built with `document.createElement()` and every value from the server is inserted with `textContent` – never `innerHTML` – so HTML in a title is shown as text. An empty result shows "No events found…", a failure an error box with a *Try again* button.

studenthub/assets/js/events-live.js (lines 73–93) – list request with loading state and cancellation

```
73      async function load() {
74          if (controller) controller.abort();
75          const current = controller = new AbortController();
76          const params = Object.fromEntries(new FormData(filters));
77          results.setAttribute('aria-busy', 'true');
78          loading.hidden = false;
79          try {
```

```
            const { status, data } = await api('list', { params, signal: current.signal });
            if (!data.ok) {
                renderError(data.error || 'The events could not be loaded (HTTP ' + status + ').');
                return;
            }
            events = data.events;
            render();
            summary.textContent = data.count === 0 ? 'No events match your search.' : data.count +
    (data.count === 1 ? ' event found.' : ' events found.');
        } catch (error) {
            if (error.name !== 'AbortError') renderError('The events could not be loaded.');
        } finally {
            if (current === controller) {                  // a newer search has not replaced this
    one
                results.setAttribute('aria-busy', 'false');
                loading.hidden = true;
```

6.4 Create and update without reloading the page

The administrator's dialog (`<dialog>` opened with `showModal()`) is filled from the event's JSON for editing. `save()` sends the form values as JSON; on HTTP 422 it writes each message from `errors` under its field and marks the field `aria-invalid`; on success it closes the dialog, shows the server's message and reloads only the list. Delete asks for confirmation and then sends `{"id": N}`; register and cancel send `{"event_id": N}` and refresh the list afterwards, so a conflict (for example a seat taken by someone else) is shown and the card is corrected at once.

studenthub/assets/js/events-live.js (lines 187–215) – create / update and field errors

```
    async function save(event) {
        event.preventDefault();
        clearErrors();
        const values = Object.fromEntries(new FormData(form));
        const id = values.id;
        delete values.id;
        const action = id ? 'update' : 'create';
        if (id) values.id = Number(id);
        const button = document.getElementById('event-save');
        button.disabled = true;
        const { status, data } = await api(action, { method: 'POST', body: values });
        button.disabled = false;
        if (status === 422 && data.errors) {
            for (const [field, text] of Object.entries(data.errors)) {
                const target = document.getElementById('f-' + field + '-error');
                if (target) target.textContent = text;
                if (form.elements[field]) form.elements[field].setAttribute('aria-invalid', 'true');
            }
            const first = form.querySelector('[aria-invalid="true"]');
            if (first) first.focus();
            return;
        }
        if (!data.ok) {
            dialog.close();
            showMessage('error', data.error);
            return;
        }
        dialog.close();
        showMessage('success', data.message);
```

6.5 AJAX search with debounce

The search box sends no request per key press. Each key press restarts a 300 ms timer; only when the user pauses is the request sent. The category, registration-status and date selects send a request at once when they change. In the Edge test, typing "workshop" quickly produced exactly one API request.

studenthub/assets/js/events-live.js (lines 227–232) – debounce wiring

```
    filters.addEventListener('submit', (e) => { e.preventDefault(); load(); });
    filters.elements.q.addEventListener('input', () => {        // debounce: one request 300 ms after
    the last key press
        clearTimeout(timer);
```

```
230          timer = setTimeout(load, 300);
231      });
232      for (const name of ['category', 'status', 'when'])
     filters.elements[name].addEventListener('change', load);
```

7. Request and Response Examples

The page sends every POST as `fetch(url, {method: 'POST', headers: {Accept: 'application/json', 'Content-Type': 'application/json', 'X-CSRF-Token': token}, body: JSON.stringify(values), credentials: 'same-origin'})`. The following exchanges happened in the Edge test. The status codes are taken from `tests/browser_api_output.txt` (for list and update, which that output does not state, from the same actions in `tests/test_api_output.txt`); the messages are the ones the page displayed from the JSON (visible in the figures of section 9).

Request	Status	Answer shown by the page
GET `api/events.php?action=list&q=&category=&status=&when=upcoming`	200	"5 events found." and the cards (Figure 2)
POST `?action=create` – location empty, capacity 0	422	Field messages under Location and Capacity (Figure 5)
POST `?action=create` – valid event	201	"Event “Hackathon Kick-off” was published." (Figure 6)
POST `?action=update` – `{"id": …, …}`	200	Card shows the new location and capacity (Figures 7–8)
POST `?action=register` – `{"event_id": …}` (student)	201	"You are registered for “Git & GitHub Workshop”." (Figure 8)
POST `?action=register` – event already full	409	"The event is full." (Figure 9)
POST `?action=delete` – event with a registration	409	"This event has 1 registration(s) and cannot be deleted, …" (Figure 10)
GET `?action=list` without a session	401	`{"ok":false,"error":"Please sign in to use StudentHub."}` (Figure 12)

Two responses were opened directly in Edge and are reproduced verbatim from the screenshots. The list response (Figure 11, `GET api/events.php?action=list&when=upcoming&category=Workshop` as administrator) begins with

`{"ok":true,"count":3,"events":[{"id":4,"title":"Robotics \u0026 Automation Exhibition","category":"Workshop","event_date":"2026-11-10","event_time":null,"when":"10 Nov 2026","location":"CSPIT Foyer","description":"","capacity":null,"seats_taken":0,"seats_left":null,"status":"open","is_past":false,"poster_url":null,"registrations_total":0,"my_registration":null}, …`

and ends with

`"filters":{"q":"","category":"Workshop","status":"","when":"upcoming"},"can_manage":true,"can_register":false}`

. The & of the title is sent as `\u0026` because of `JSON_HEX_AMP`. A visitor without a session receives `{"ok":false,"error":"Please sign in to use StudentHub."}` with HTTP 401 (Figure 12). All status codes of the API, from the source code:

Status	When
200 / 201	list, get, update, delete, cancel / create, register succeeded (`{"ok": true, …}`)
400	Unknown action, malformed JSON, a JSON array, a body over 20,000 bytes

401	No valid session (not signed in, timed out, deactivated student)
403	Missing or wrong `X-CSRF-Token`; the role lacks the permission; unlinked account or administrator registering; inactive student
404	Unknown or malformed event ID
405	Wrong method, with an `Allow` header
409	Duplicate, full, closed or past event, nothing to cancel, delete of an event with registrations
415	POST without `Content-Type: application/json`
422	Validation failed – `{"ok": false, "error": …, "errors": {field: message}}`
500 / 503	Unexpected server error / database unavailable – general message only, details in the server log

8. Validation and Security

- **Server-side validation** of every field with the portal's `validate_event()` before any INSERT or UPDATE (422 with one message per field); the dialog's HTML attributes are only a convenience.
- **SQL injection:** MySQLi prepared statements in the shared include files; the API has no own connection and no `->query()` call (static test); filter values only from fixed lists; `%` and _ searched literally.
- **Safe output:** JSON encoded with the `JSON_HEX_*` flags; the page inserts data only with `textContent`; posters only as `poster.php` URLs.
- **CSRF:** POST requires `application/json` and the session's token in `X-CSRF-Token` (compared with `hash_equals()`); failures are audit-logged.
- **Correct methods:** reading only with GET, changing only with POST; 405 with an `Allow` header otherwise.
- **Authentication and roles:** JSON 401 without a session; `events.manage` for create / update / delete; only a linked student account can register; ACCESS_DENIED logged for refused roles. The page's buttons follow `can_manage` / `can_register`, but the server checks every request.
- **Integrity:** the registration transaction of Practical 12 (row lock, capacity, duplicates) – 6 simultaneous API registrations for 3 seats gave exactly three 201 and three 409.
- **Central error handling:** one exception handler; database and PHP errors are logged, the client receives a general message; an unreachable database gives HTTP 503 in JSON.

9. Output Screenshots

All screenshots are genuine full-page captures from Microsoft Edge (headless, driven by Playwright, window 1280 px wide) taken by `tests/browser_api.py` while the portal ran on PHP's built-in server with a disposable database built from `database/studenthub_demo.sql` plus four fictional events and two fictional students added by the test. The test counted page navigations: create, update and delete happened without a page reload. Captures 05 (the filled dialog before saving) and 09 (the card after the update, also visible in Figures 8–10) are not reproduced. In the full-page captures of the modal dialog (Figures 5 and 7) the darkened backdrop covers only the 900-pixel-high browser window, not the whole capture.

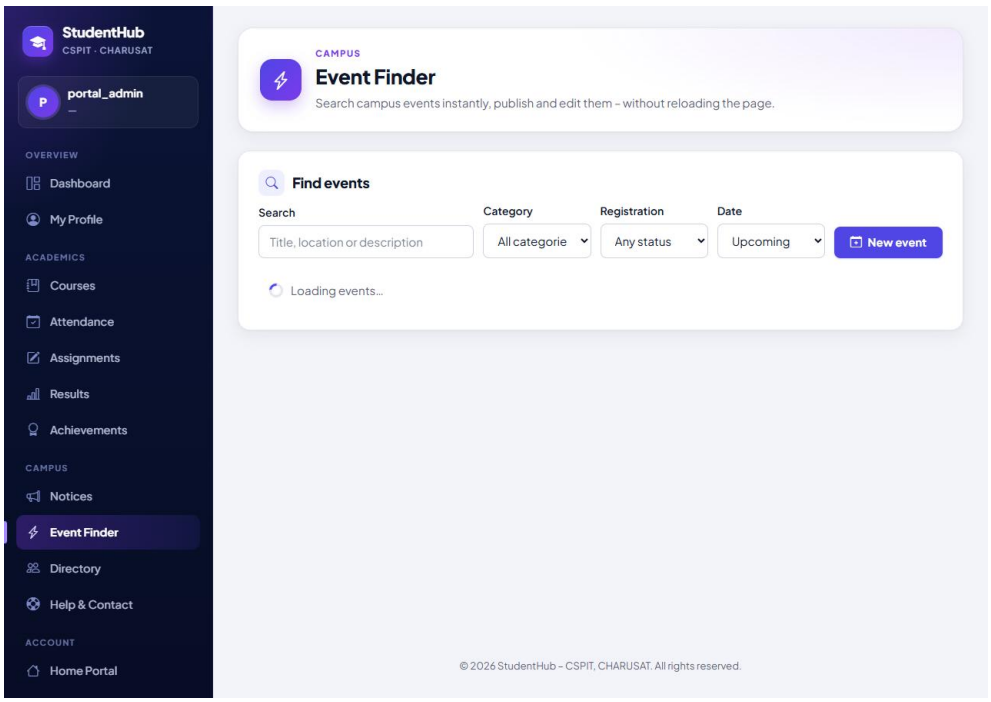


Figure 1: Loading state of the Event Finder (administrator portal_admin): "Loading events…" with a spinner below the filters while the first list request runs. For this screenshot the test held the API answer back for 1.5 s in its own network layer (a Playwright route); the answer itself was the real one (01-loading-state.png)

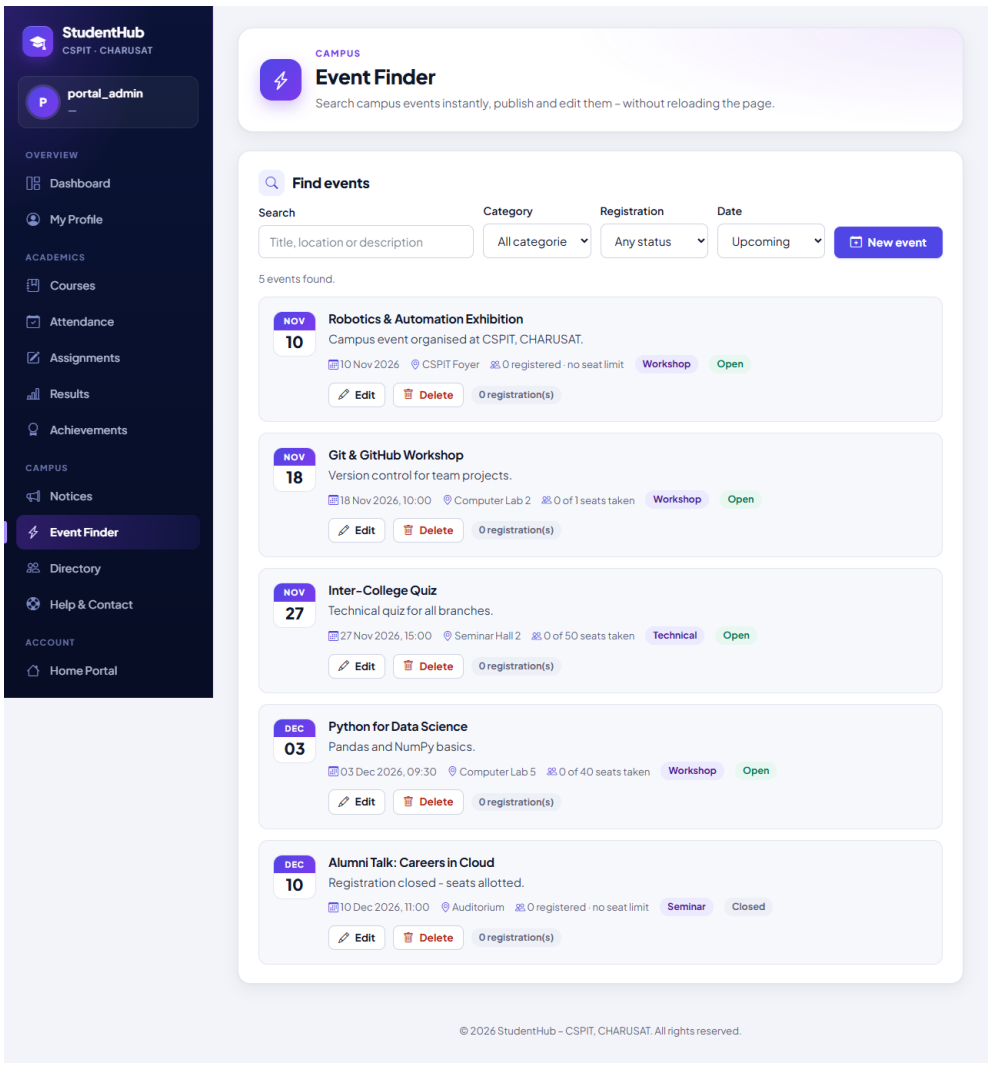


Figure 2: Upcoming events loaded from the JSON API, soonest first: "5 events found." – each card shows date, location, seats, category and Open / Closed badge and, for the administrator, Edit, Delete and the number of registrations (02-event-list.png)

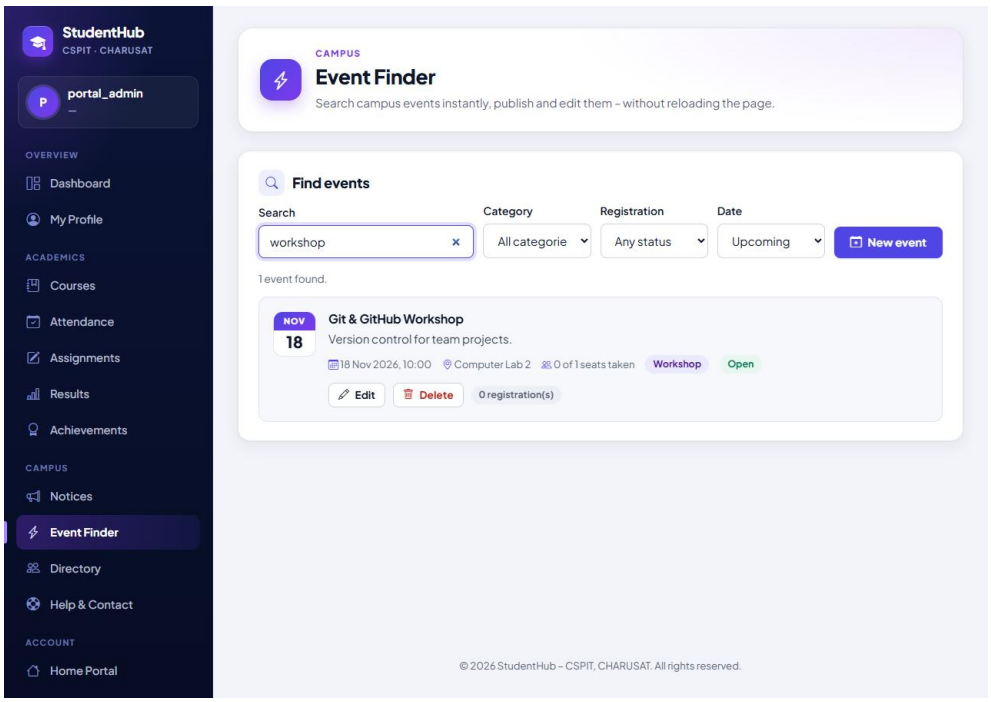


Figure 3: Debounced live search: typing "workshop" sent a single API request (counted by the test); "1 event found." – Git & GitHub Workshop (03-live-search.png)

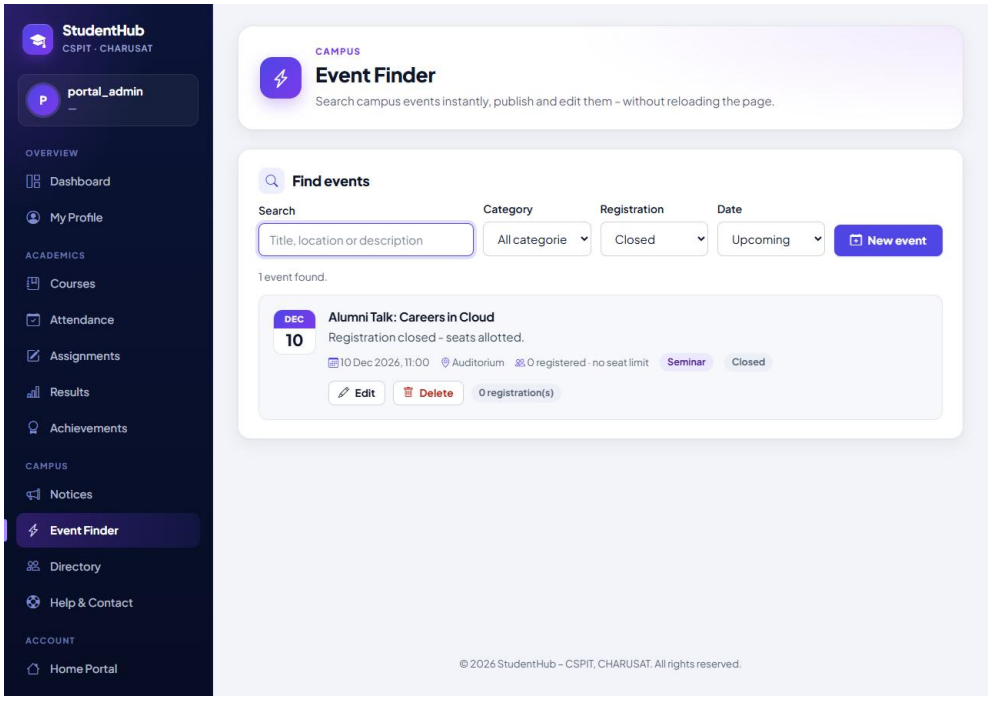


Figure 4: Registration filter "Closed" applied through the API: "1 event found." – Alumni Talk: Careers in Cloud (04-filters.png)

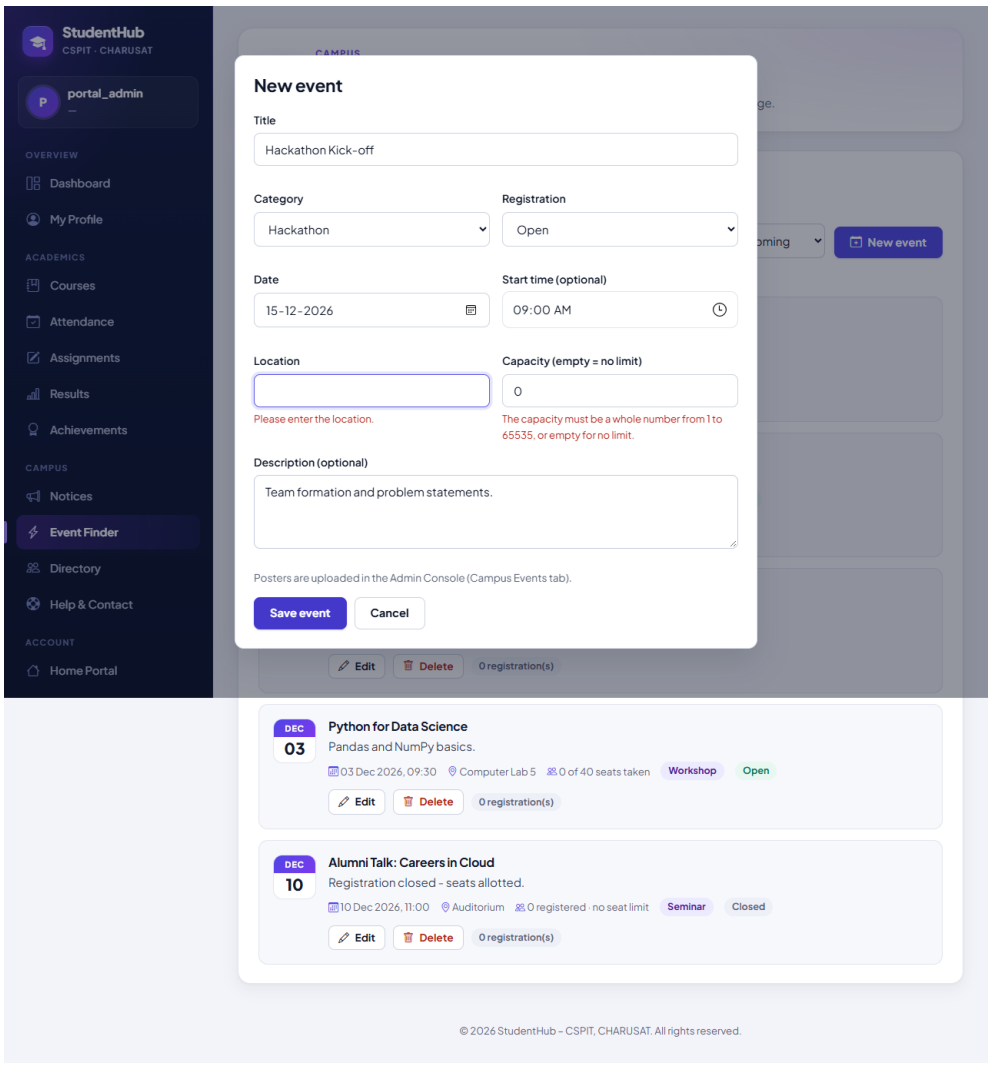


Figure 5: New-event dialog after Save with an empty location and capacity 0: the API answered HTTP 422 and the page shows "Please enter the location." and "The capacity must be a whole number from 1 to 65535, or empty for no limit." under the fields; nothing was stored (06-validation-error.png)

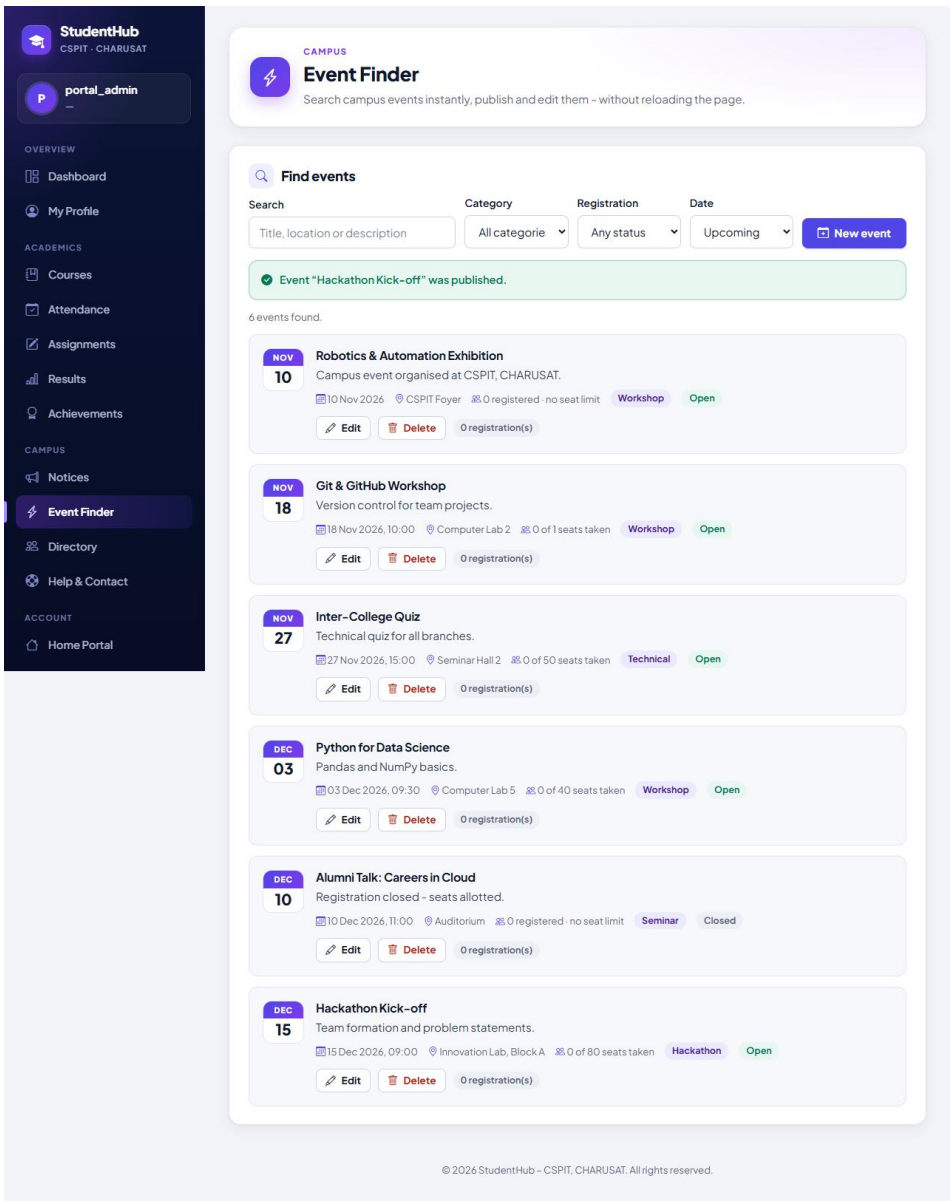


Figure 6: Create via fetch (HTTP 201) without a page reload: "Event “Hackathon Kick-off” was published." – 6 events found; the new event (15 Dec 2026, 09:00, Innovation Lab, Block A, 0 of 80 seats taken) is in the list (07-event-created.png)

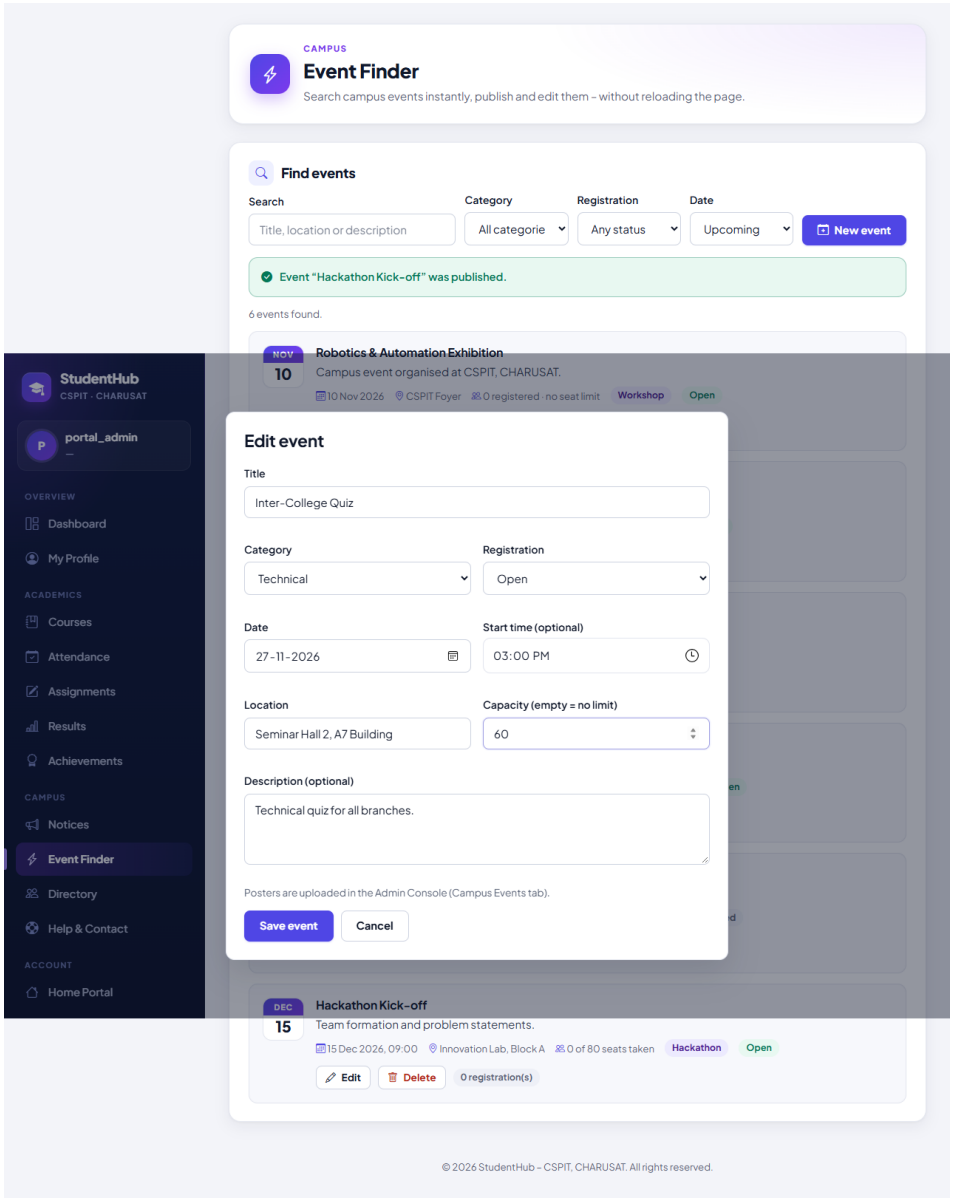


Figure 7: Edit dialog for "Inter-College Quiz" filled from the JSON data: location changed to "Seminar Hall 2, A7 Building" and capacity to 60 before saving (08-edit-event.png)

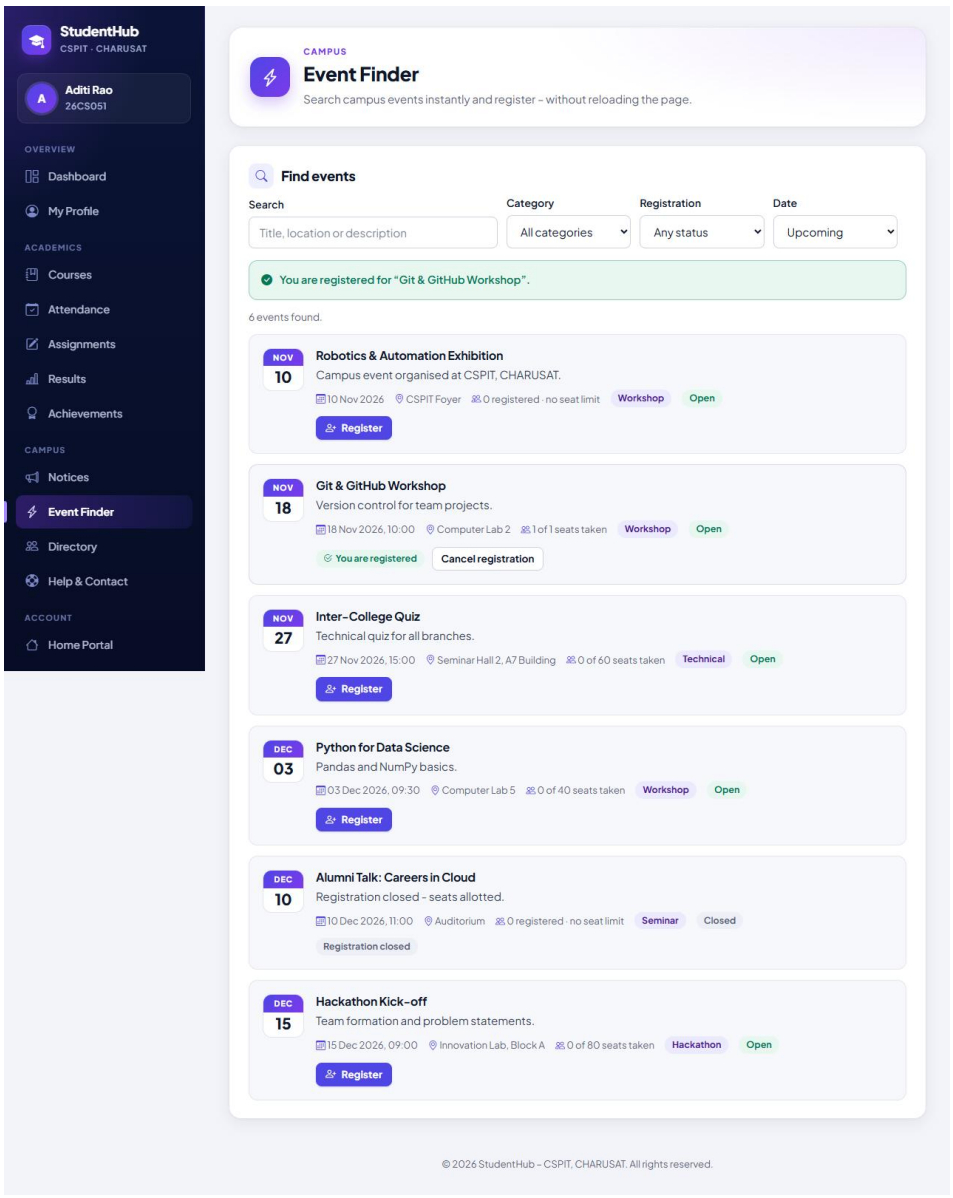


Figure 8: The student Aditi Rao (26CS051) registers via fetch (HTTP 201): "You are registered for "Git & GitHub Workshop"." – the card shows "You are registered", Cancel registration and 1 of 1 seats taken; the updated Inter-College Quiz shows Seminar Hall 2, A7 Building and 0 of 60 seats taken (10-student-registered.png)

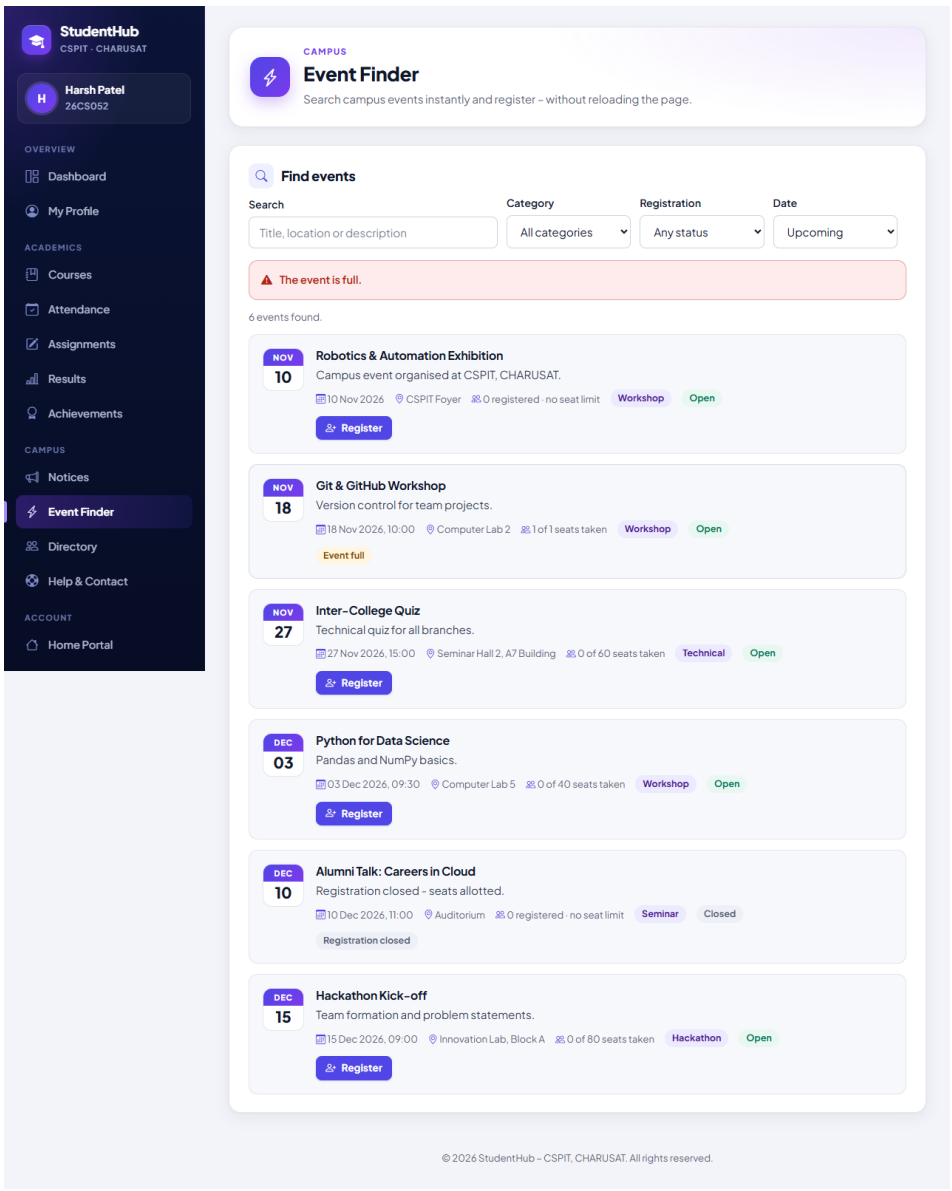


Figure 9: The student Harsh Patel (26CS052) clicks the Register button that was still shown on his page after the last seat had been taken: the API answers 409 and the page shows "The event is full."; the refreshed list now shows "Event full" (1 of 1 seats taken) (11-registration-conflict.png)

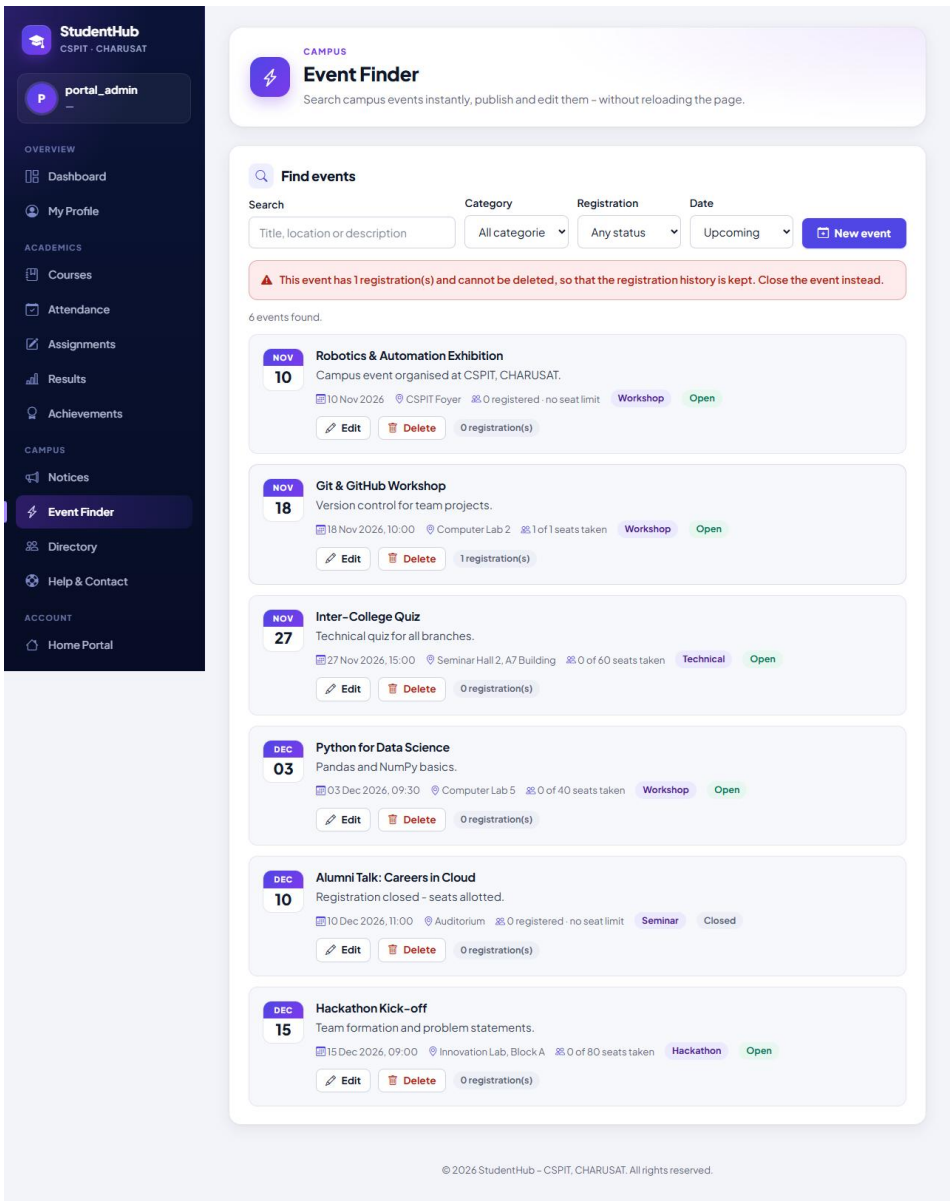


Figure 10: The administrator deletes "Git & GitHub Workshop", which has 1 registration: the API answers 409 – "This event has 1 registration(s) and cannot be deleted, so that the registration history is kept. Close the event instead." – and the event stays in the list (12-delete-conflict.png)

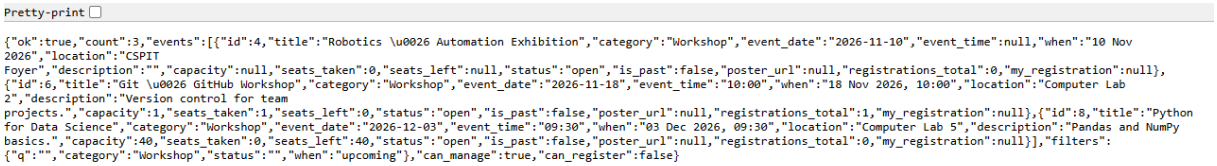


Figure 11: Raw JSON of api/events.php?action=list&when=upcoming&category=Workshop opened in Edge as the administrator: {"ok":true,"count":3,…} with exactly the documented event fields, "&" encoded as \u0026, the active filters and can_manage true / can_register false (13-api-json-response.png)



Figure 12: A visitor without a session calling api/events.php?action=list: HTTP 401 with the JSON body {"ok":false,"error":"Please sign in to use StudentHub."} – no redirect to the login page (14-api-unauthorised.png)

10. Testing and Results

All tests were executed on 31 October 2026 on Windows 11 with PHP 8.2.12 (XAMPP), MariaDB 10.4.32 and PHP's built-in server, against disposable databases; a second test server was pointed at a database that does not exist to check the HTTP 503 answer. The numbers are copied from the saved outputs and from TEST_REPORT.md §10.

Suite	Result	Output
`tests/test_api.py` – HTTP tests of `api/events.php` and `events_live.php`	**72 / 72 passed**	`tests/test_api_output.txt`

`tests/browser_api.py` – Microsoft Edge + screenshots	**16 / 16 passed**	`tests/browser_api_output.txt`
Regression: `test_events` 94 / 94, `test_students` 170 / 170, `test_auth` 132 / 132, `test_001` 88 / 88	all passed	outputs in `tests/` and `database/tests/`
Regression in Edge: `browser_events` 16 / 16, `browser_students` 16 / 16, `browser_inactive_login` 13 / 13, `browser_auth` 14 / 14	all passed	outputs in `tests/`

10.1 Checks per area (tests/test_api.py)

Area	Checks	Examples of what is verified
Static checks	4	`php -l`; portal connection and helpers only, no `->query()`; `textContent` only and `X-CSRF-Token`; 300 ms debounce
Authentication	2	not signed in: list and create → 401 JSON, no redirect, nothing stored
GET list / get	19	response shape and exact event fields, role flags, filters and combinations, `%` _ and SQL strings as text, HTML in a title escaped as `\u003C`, 404 for unknown / malformed IDs
Methods, content type, CSRF, format	6	405 with `Allow`, 400 unknown action, 403 for a missing / wrong / modified token (logged), 415 for a form-encoded POST, 400 for malformed JSON, an array, a body over 20 kB
Create / update / delete	13	create 201 + CREATE_EVENT, eight invalid bodies → 422 with `errors`, update 200 / 404 / 422
Register / cancel and conflicts	17	register 201, duplicate / full / closed / past 409, unknown 404, cancel 200 / 409, unlinked account and admin 403, 6 parallel registrations for 3 seats, delete 409 / 200 / 404
Roles	3	students and editors 403 for create / update / delete (ACCESS_DENIED); a deactivated student loses API access (401)
Event Finder page	5	student, admin, editor and visitor views of `events_live.php`; no PHP warnings
Database unavailable / clean-up	3	HTTP 503 JSON with a general message only; disposable database removed; live database untouched
Total	**72**	

The Edge test (16 checks) verified the loading state and its disappearance, the list from the API, the debounced search (one request), the filters, the 422 field messages, create / update / delete via fetch without a page reload, a student registration (201), the 409 for a stale Register button with the list refreshed to *Event full*, the 409 for deleting an event with a registration, the raw JSON response, the 401 for a visitor, no JavaScript errors other than the intended 401 / 409 / 422 responses and no PHP warnings or API errors (apart from the known 404 for `/favicon.ico`).

11. Problems Found and Fixed During Integration

- **Loading indicator stayed visible (fixed):** the browser test found that the page's `display: flex` rules for `.live-loading` and `.live-message` overrode the `hidden` attribute, so "Loading events…" stayed on screen after the list had loaded. An explicit rule now hides both elements while they carry `hidden`:

studenthub/assets/css/global.css (lines 458–459) – the fix for the hidden attribute

```
458  .live-loading { display: flex; align-items: center; gap: 10px; padding: 14px; color: var(--text-
     muted); }
459  .live-loading[hidden], .live-message[hidden] { display: none; }        /* display: flex must not
     override the hidden attribute */
```

- **Redirect versus JSON:** the portal's `require_login()` redirects to the login page, which `fetch()` cannot use. The API calls `validate_session()` itself and answers 401 JSON (section 6.2); with `STUDENTHUB_API` defined, `db.php` also answers an unreachable database with HTTP 503 JSON instead of an HTML page.
- **Stale buttons:** a Register button can be outdated when someone else takes the last seat. The server refuses the request (409) and the page refreshes the list after every register / cancel, so the card shows *Event full* at once (Figure 9).
- **Evidence of the loading state:** the real API answers within milliseconds, so the test delayed the answer in its own network layer for screenshot 01; the response itself was not changed.

12. Differences from the Original Standalone Practical and Limitations

Aspect	Original report (Practical-13/)	Integrated portal (studenthub/)
Database	Own `studenthub_prac13` with its own events table (title 3–120, description 10–1000, venue, capacity 1–5000)	Portal `studenthub_db.events`, shared with the Admin Console and the Campus Events page; no schema change
Files	`api.php`, `index.html`, `script.js`, `db.php`, `schema.sql`	`api/events.php`, `events_live.php`, `assets/js/events-live.js`, portal `db.php`
Request protection	JSON body and `X-Requested-With: fetch`	Portal session, `application/json` and the session's token in `X-CSRF-Token`
Access	–	Roles: administrators manage, students register / cancel, editors read; 401 JSON without a session
Validation	Own `validate_event()` in api.php	The portal's `validate_event()`, the same rules as the Admin Console
Debounce	350 ms + AbortController	300 ms + AbortController
Registration	–	register / cancel with the Practical 12 transaction (409 conflicts)
Delete	Any event	Refused (409) for events with registrations
Network evidence	Edge DevTools Network panel screenshots	Raw JSON opened in Edge (Figures 11–12); the test counts API requests and page navigations. No DevTools screenshots were taken for the integrated version
Tests	89 / 89 API, 13 / 13 browser	72 / 72 API, 16 / 16 browser

Limitations (TEST_REPORT §10): posters are uploaded only in the Admin Console (the API handles JSON, not files); the Event Finder lists at most 100 events per request. The page needs JavaScript – without it a `<noscript>` note points to the Campus Events page, which works without JavaScript.

13. Conclusion

The Event Finder lists, searches, adds, updates and deletes events entirely with the Fetch API: the page never reloads, `api/events.php` answers every request in JSON with a fitting status code, GET and POST are separated, and all input is validated by the same rules as in the Admin Console before a prepared statement touches the database. Integrated into the portal, the API uses the portal session, roles and CSRF token, lets students register and cancel through the same overbooking-safe transaction, and answers without a session with JSON 401 instead of a redirect. Loading, success, empty and error states keep the user informed, and the debounced search sends one request per pause instead of one per key press. 72 automated API checks and 16 browser checks passed.

14. References

- Project repository: https://github.com/TheEmperor08/StudentHub-ITUE203-25CS023 – `studenthub/api/events.php`, `studenthub/events_live.php`, `studenthub/assets/js/events-live.js`, `tests/`, `evidence/practical-13-integration/`, `TEST_REPORT.md` §10; commit `a9d4e22`
- MDN – Using the Fetch API: https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch
- MDN – AbortController: https://developer.mozilla.org/en-US/docs/Web/API/AbortController
- MDN – The HTML dialog element: https://developer.mozilla.org/en-US/docs/Web/HTML/Element/dialog
- MDN – Node.textContent: https://developer.mozilla.org/en-US/docs/Web/API/Node/textContent
- PHP Manual – json_encode: https://www.php.net/manual/en/function.json-encode.php
- PHP Manual – MySQLi prepared statements: https://www.php.net/manual/en/mysqli.quickstart.prepared-statements.php
- OWASP – REST Security Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/REST_Security_Cheat_Sheet.html